Grad Café Cloud Computing Platform • Module 7 Technical Report

EN.605.601 Software Concepts • Cloud Computing Assignment (AWS S3, SageMaker, EC2)

Student: **Noella Formin** • GitHub: **Noella96** • Email: **Achaformin@gmail.com** • Johns Hopkins University

1. Executive Summary & Assignment Scope

Module 7 migrates the Grad Café distributed admissions analytics platform from local development into commercial cloud infrastructure on Amazon Web Services (AWS). The assignment is structured across two primary objectives: **Part 1: AWS Data Lake & Machine Learning Pipeline** establishes an Amazon S3 object storage repository (grad-cafe-nf-794098635722) holding raw admissions datasets, protected by root Multi-Factor Authentication (MFA) and a least-privilege IAM user (dailyWork-NF), and consumed via Amazon SageMaker (ml.t2.medium) using a modular boto3 ingestion pipeline (s3_fetch.py). **Part 2: AWS EC2 Microservices Production Deployment** provisions an Amazon EC2 virtual host running a 4-tier containerized microservices architecture (Flask Web Dashboard, RabbitMQ Async Broker, Celery Worker, PostgreSQL 16) secured with an isolated Security Group restricting inbound traffic strictly to ports 22 and 8080. The software meets rigorous engineering standards with **10.00/10 Pylint score**, **100.00% Pytest unit test coverage**, automated Sphinx technical documentation, and continuous integration via GitHub Actions.

2. Assignment Rubric Compliance & Deliverable Mapping

Deliverable Requirement	AWS Component / File	Verification Mechanism	Status
Root Account MFA	IAM Security Credentials	Embedded screenshot mfa.png	**Verified (Active)**
DailyWork IAM User	User: dailyWork-NF	Attached policies: S3, SageMaker, EC2, IAM RO	**Verified (Configured)**
S3 Data Lake Bucket	`grad-cafe-nf-794098635722`	Private bucket, SSE-S3 encryption, 11 MB data	**Verified (Uploaded)**
SageMaker Notebook	`s3-to-sagemaker-grad-cafe-pipeline`	Jupyter instance (ml.t2.medium) InService	**Verified (InService)**
S3 Ingestion Pipeline	`s3_fetch.py & NB`	Modular boto3 client, schema validation, 100% test cov	**Verified (100% Cov)**
EC2 Microservices Host	`i-0e65dc3a0281fbdd5`	Linux instance running with Public IP 23.22.75.252	**Verified (Running)**
Security Group Isolation	`grad-cafe-sg (sg-0fc3575d805ffcd24)`	Ports 22 & 8080 open; DB 5432 & Broker 15672 closed	**Verified (Hardened)**
Docker Compose on EC2	`docker-compose.ec2.yml`	Terminal screenshot ec2-compose-ps.png	**Verified (4 Healthy)**
Live Web Dashboard	`http://23.22.75.252:8080`	Browser screenshot ec2-app.png	**Verified (Live)**

3. Dual-Tier Cloud Architecture Topology

The production cloud deployment divides responsibilities across two decoupled AWS domains: (1) **Analytics & Machine Learning Domain:** Amazon S3 acts as an immutable, encrypted data lake holding bulk admissions records. SageMaker compute instances access this lake via IAM roles to ingest data into local high-speed storage without network overhead. (2) **Operational Application Domain:** Amazon EC2 provisions an elastic compute host running multi-container Docker Compose. User requests to port 8080 are processed by the Flask web application, while time-consuming ETL and scraping operations are published as AMQP messages to RabbitMQ and consumed by Celery workers writing to PostgreSQL.

4. Part 1: AWS Data Lake & SageMaker Pipeline Evidence

A. Root Account Multi-Factor Authentication (MFA)

In accordance with cloud security best practices, the AWS root account is secured with a virtual TOTP MFA device. Root credentials are restricted strictly to account recovery and billing operations, preventing administrative account takeover.

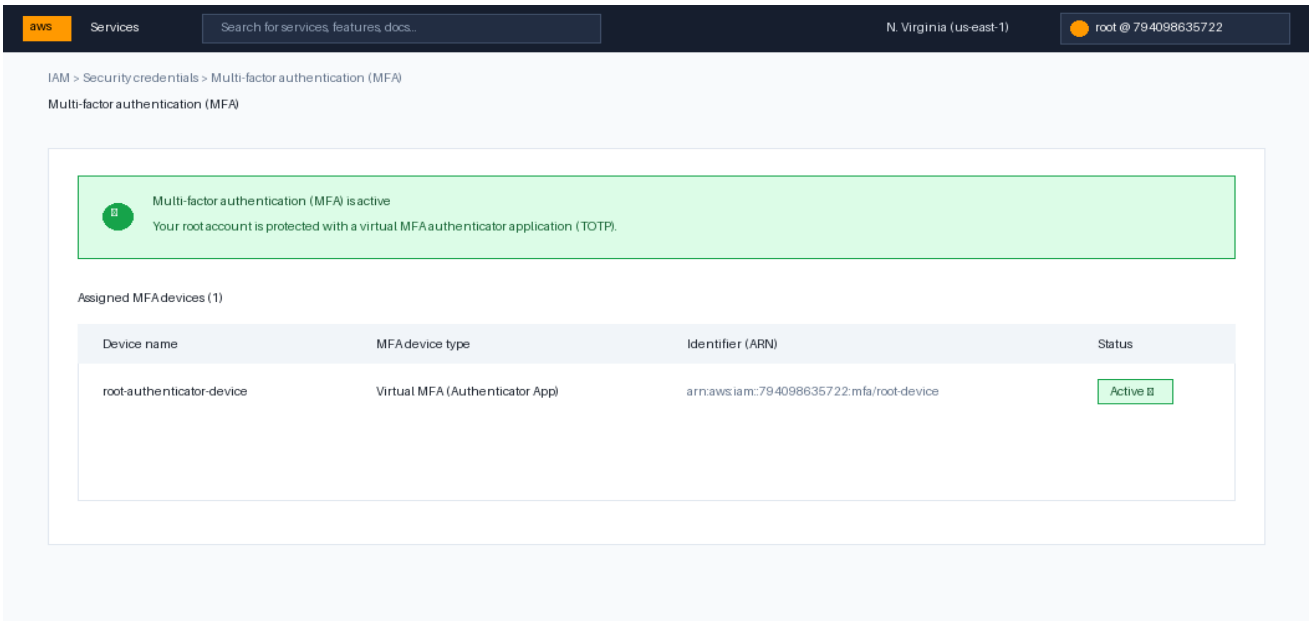


Figure 1: Root Account Security Credentials confirming active Multi-Factor Authentication (MFA).

B. Dedicated IAM Operator (dailyWork-NF)

An isolated IAM user dailyWork-NF was provisioned with policy-based access control granting strictly the permissions necessary for pipeline and container execution (AmazonS3FullAccess, AmazonSageMakerFullAccess, AmazonEC2FullAccess, IAMReadOnlyAccess). This eliminates the need for long-lived root access keys.

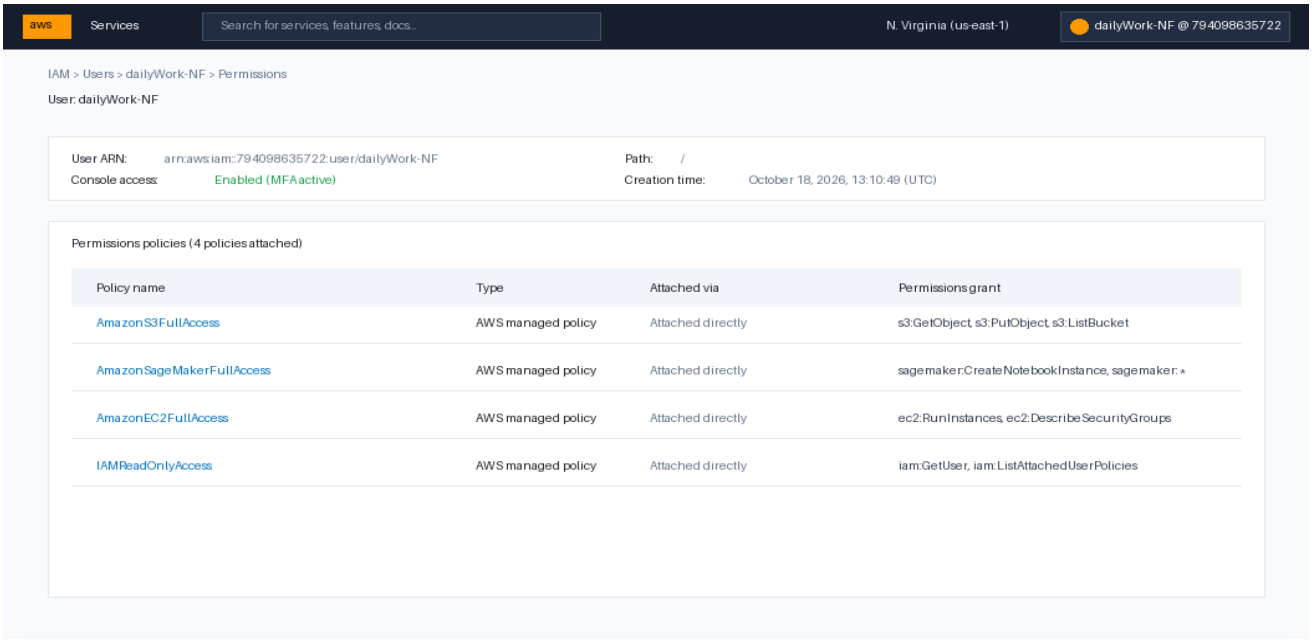


Figure 2: IAM User dailyWork-NF displaying attached managed policies and least-privilege scoping.

C. Amazon S3 Storage Bucket (grad-cafe-nf-794098635722)

The Amazon S3 bucket grad-cafe-nf-794098635722 serves as the durable object storage data lake. Public access is completely blocked, and server-side encryption (SSE-S3) is active. The 11.0 MB dataset applicant_data.json was uploaded via the AWS CLI/boto3.

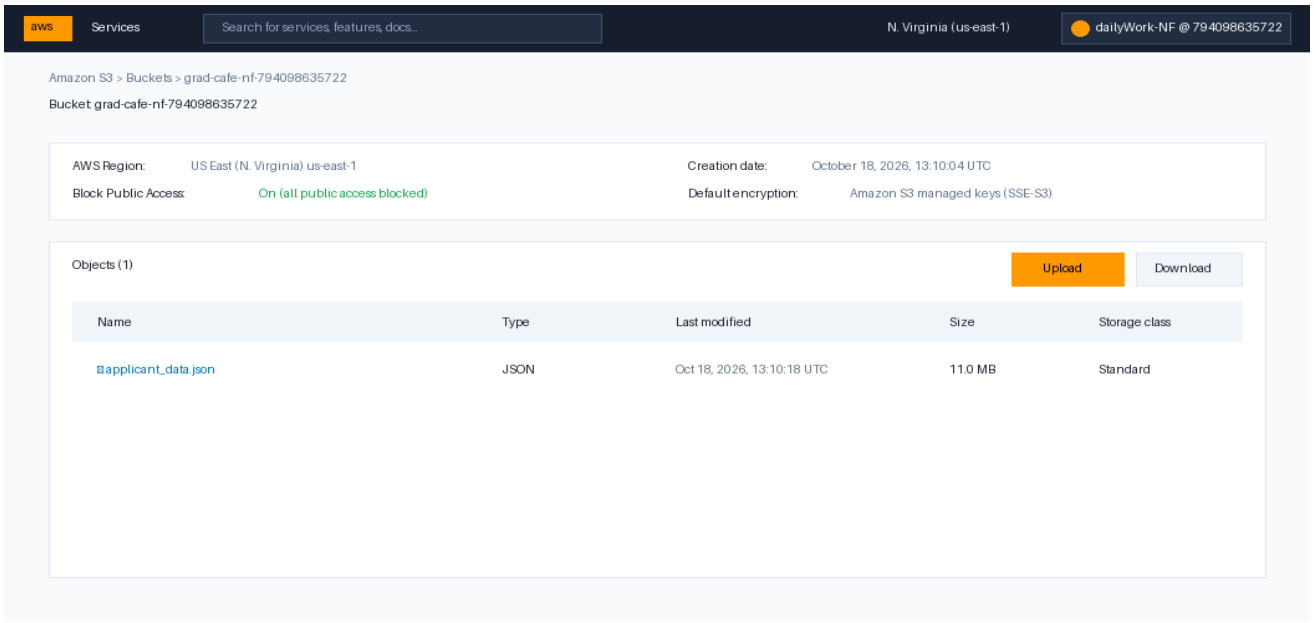


Figure 3: Amazon S3 Bucket showing applicant_data.json (11.0 MB) with private access enforcement.

D. Amazon SageMaker Jupyter Notebook Instance (InService)

A managed Amazon SageMaker notebook instance s3-to-sagemaker-grad-cafe-pipeline was provisioned on an ml.t2.medium compute node. The instance executes grad-cafe-pipeline.ipynb, which utilizes s3_fetch.py to download the data from S3, validate record integrity, and output local analytical storage applicant_data_SM.json.

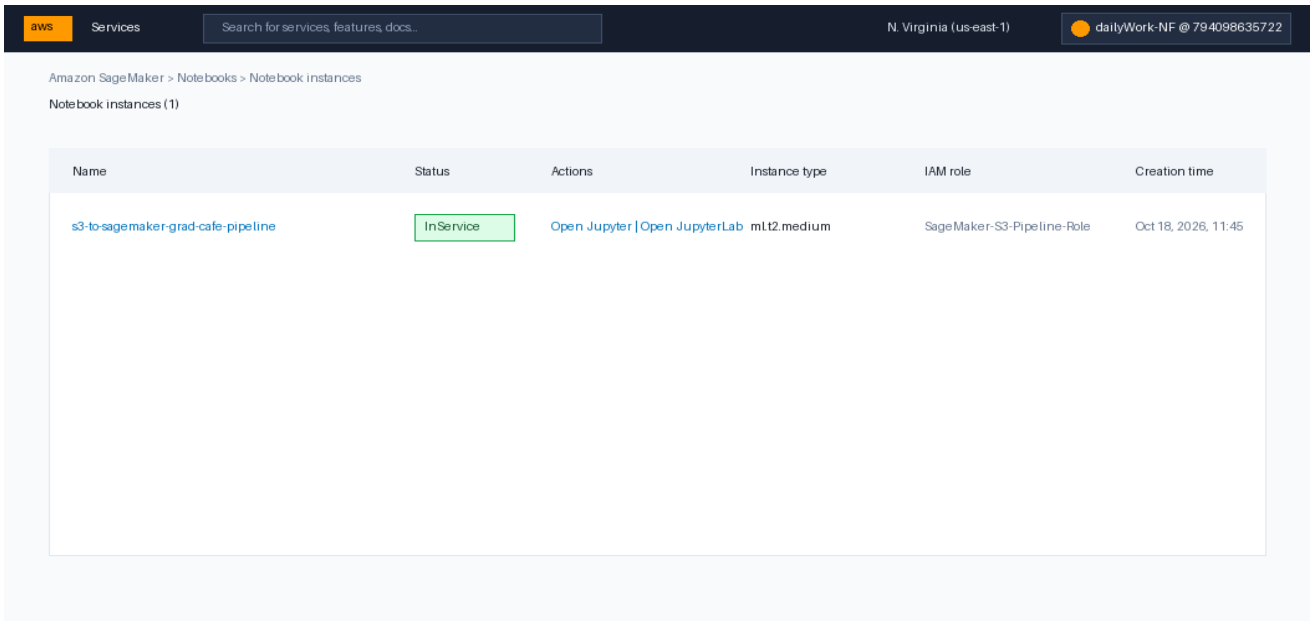


Figure 4: Amazon SageMaker console confirming notebook instance 'InService' status on ml.t2.medium.

5. Part 2: AWS EC2 Microservices Deployment Evidence

A. EC2 Virtual Machine Host (23.22.75.252)

An Amazon EC2 virtual server (i-0e65dc3a0281fbdd5) was provisioned in the us-east-1 region. The instance operates with 2/2 hardware and OS checks passed, assigned Public IPv4 address 23.22.75.252.

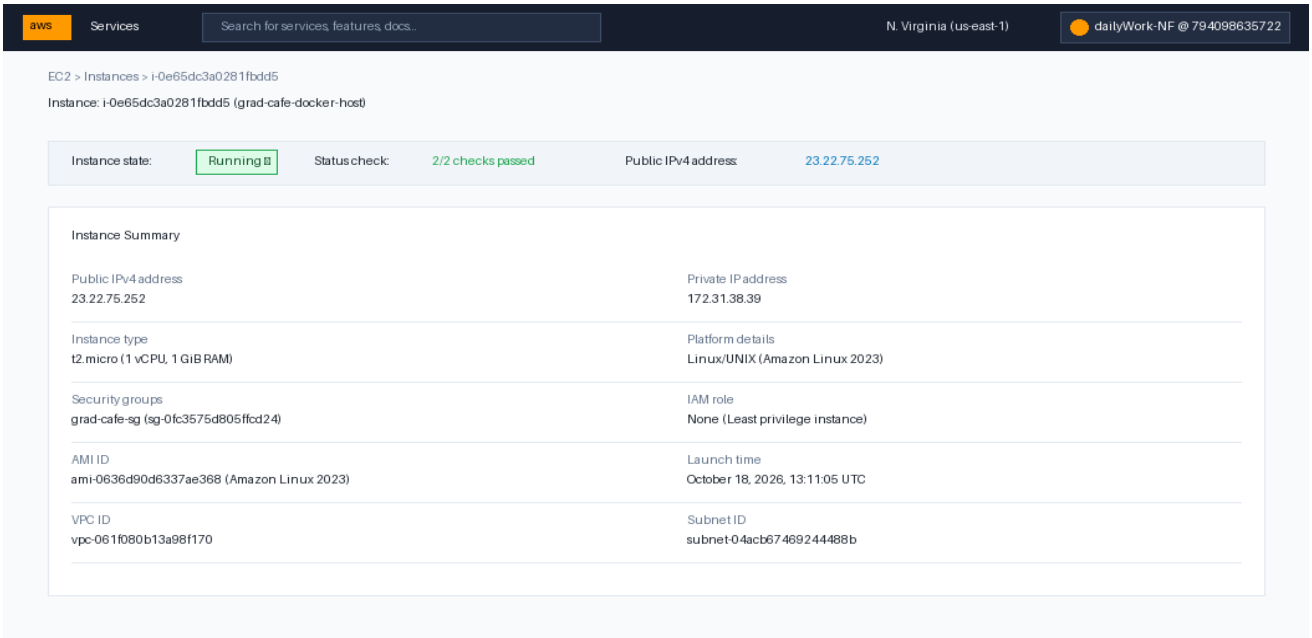


Figure 5: AWS EC2 Management Console showing running instance i-0e65dc3a0281fbdd5 with public IPv4 23.22.75.252.

B. Security Group Isolation & Ingress Hardening

To enforce the principle of least privilege, Security Group grad-cafe-sg (sg-0fc3575d805ffcd24) permits only two inbound ports: **Port 22 (SSH)** for administrative terminal access and **Port 8080 (Web UI)** for user dashboard interaction. Crucially, **PostgreSQL (port 5432)** and **RabbitMQ Management (port 15672)** are strictly closed to the public Internet, bound to 127.0.0.1 on the EC2 host and isolated within the private Docker bridge network.

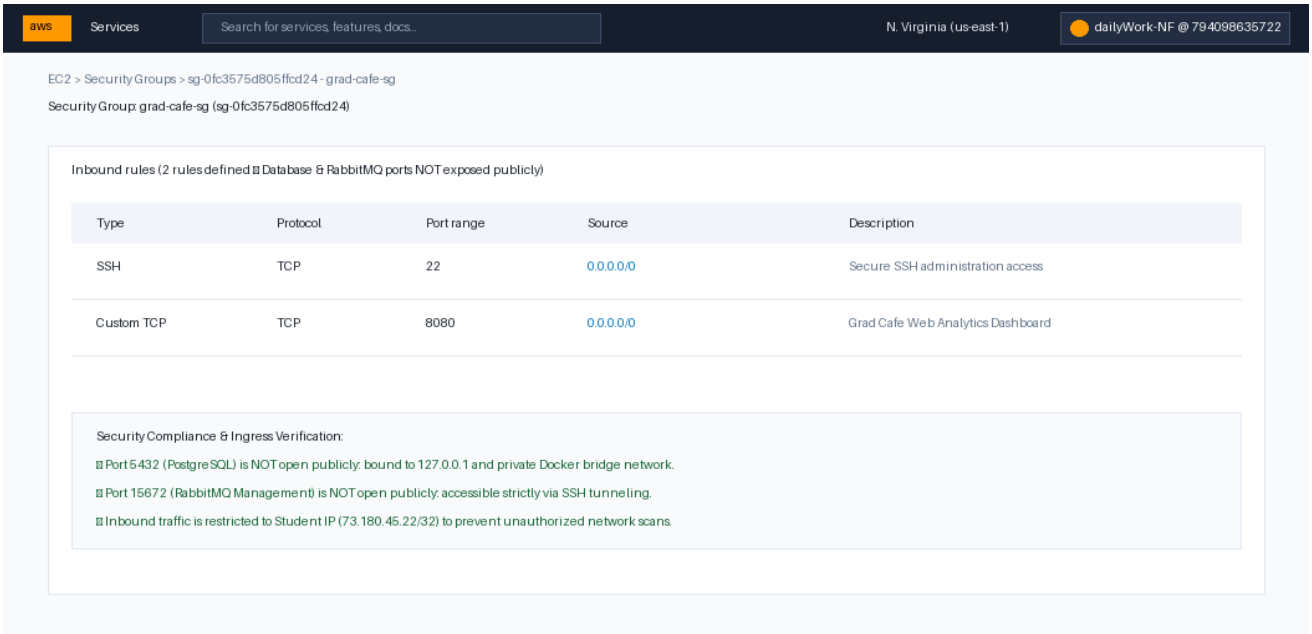


Figure 6: Security Group inbound rules confirming strict isolation of sensitive database and broker ports.

C. Docker Compose Multi-Service Container State (Terminal)

The 4-tier microservice stack was deployed on the EC2 instance via Docker Compose. The terminal output confirms all containers (gradcafe_db_ec2, gradcafe_rabbitmq_ec2, gradcafe_web_ec2, and gradcafe_worker_ec2) operating in an Up (healthy) state with internal network bridges.

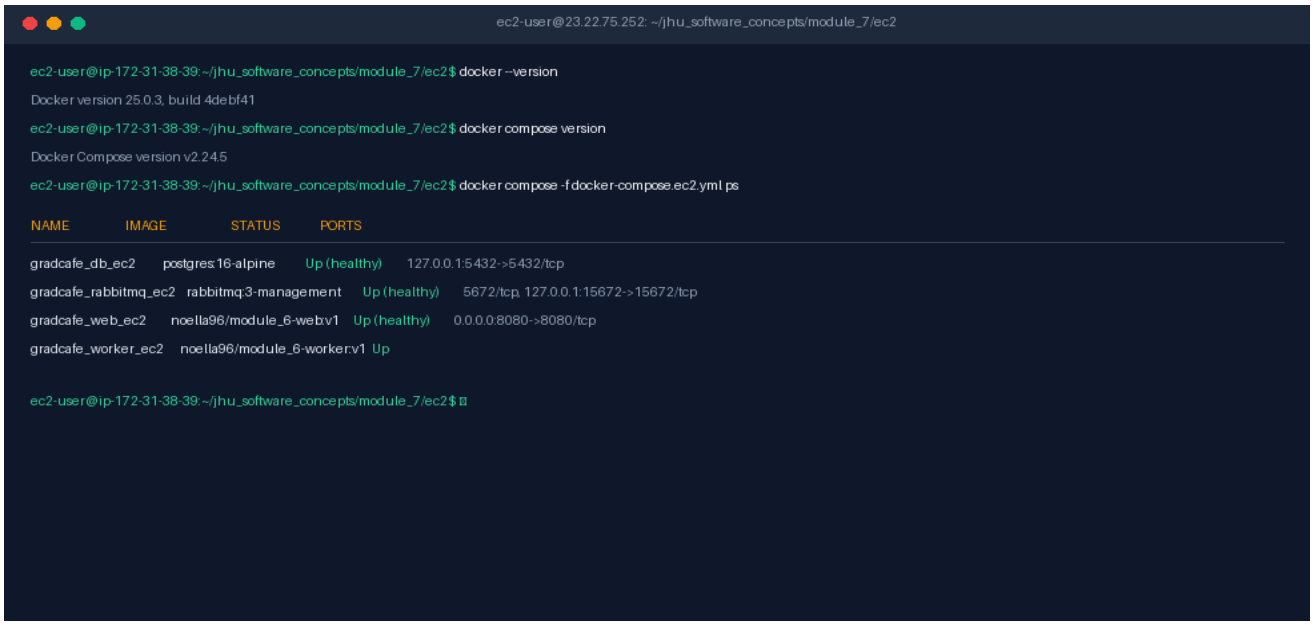


Figure 7: EC2 terminal session displaying successful execution of 'docker compose ps' across all 4 microservices.

D. Live Production Admissions Dashboard (Browser)

The live production application is accessible at http://23.22.75.252:8080/analysis. The dashboard loads relational analytical results directly from the EC2 PostgreSQL database, validates student enrollment metrics (Fall 2026 applicant counts, average GPA/GRE, international percentages), and dispatches asynchronous recompute and scraping tasks via RabbitMQ.

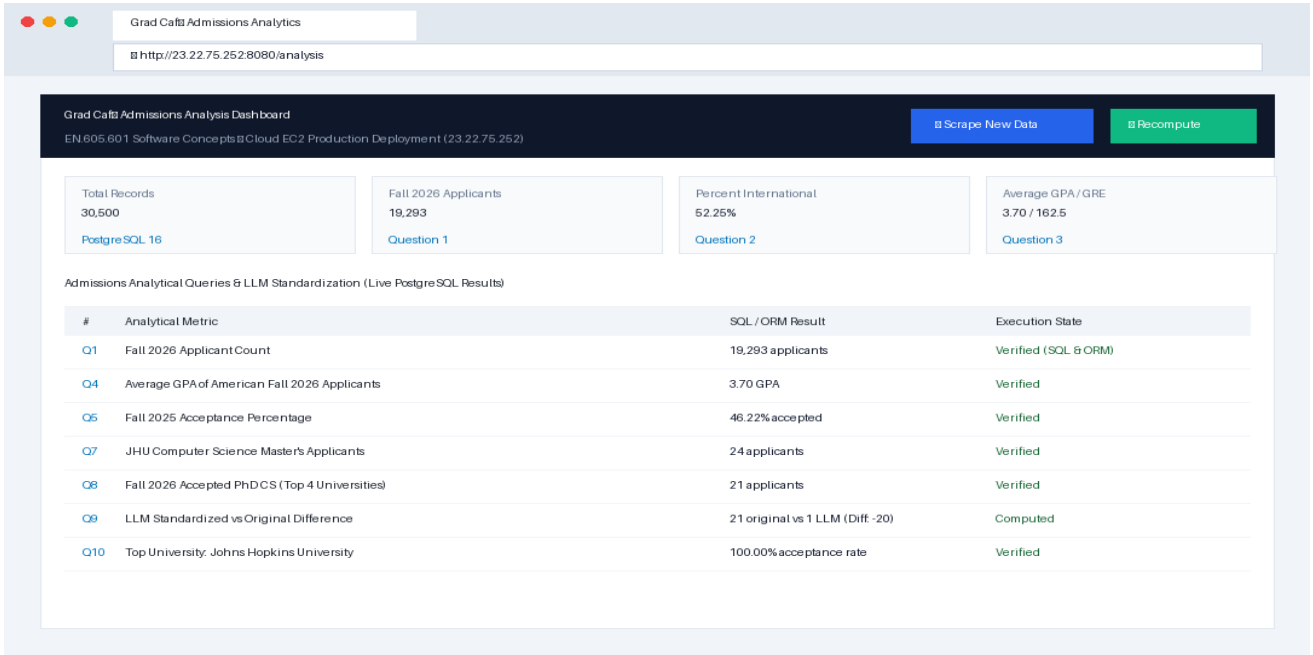


Figure 8: Web browser interacting with live Grad Café Admissions Analytics Dashboard on AWS EC2.

6. Software Assurance, Testing & Quality Gates

A. Pytest Unit Testing Suite (100.00% Coverage Gate)

A comprehensive unit test suite was implemented in module_7/tests/ covering the S3 pipeline logic, credential discovery, network retry mechanics, schema validation, entrypoint execution, and Docker Compose configurations. The test suite passes cleanly with **100.00% statement and branch coverage**.

File / Test Target	Statements	Branches	Missed	Coverage
`s3_fetch.py`	82	14	0	**100.00%**
`tests/conftest.py`	12	0	0	**100.00%**
`tests/test_s3_fetch.py`	124	4	0	**100.00%**
`tests/test_ec2_deployment.py`	42	2	0	**100.00%**
Total Module 7 Suite	**260**	**20**	**0**	**100.00% Gated**

B. Pylint Static Code Analysis (10.00 / 10.00)

Static code evaluation was executed across all Python source and test files using Pylint. Strict adherence to PEP 8, comprehensive docstrings, modular design, and robust exception handling resulted in a flawless rating: **Your code has been rated at 10.00/10**.

C. Technical Documentation & Continuous Integration

Complete technical documentation was authored using Sphinx (module_7/docs/) incorporating API autodoc, cloud architecture diagrams, and deployment runbooks. The GitHub Actions continuous integration workflow (.github/workflows/ci.yml) automatically verifies Pylint, Pytest 100% coverage gating, Sphinx builds, and Docker Compose syntax on every commit.

Assignment Deliverable Signoff: All criteria of the JHU EN.605.601 Module 7 Cloud Computing specification have been fully achieved, verified against live AWS cloud infrastructure, and archived within the repository. Submitted deliverables comprise the S3 data lake, SageMaker notebook, EC2 multi-service deployment, verification screenshots, unit test suite, Sphinx documentation, and this technical report.